

Password Manager

Final Presentation

Group 3
A01166162 Hazel Jeong
A01411168 Shinyoung You
A01418439 Victor Lum

Apr 14

App Overview & Purpose

- Purpose: the Password Manager app allows users to easily manage **many passwords** that are used in **different services**.
  - People tend to reuse the same password across different services for their convenience. But this is a very common security risk (e.g. credential stuffing).
  - So we created this app so that people follow secure password practice.
- This app handles the entire process to **evaluate**, **generate**, and **manage** passwords.
- This app is currently **publicly available** using **AWS** cloud infrastructure.

Functionalities

1. Password Evaluator

Evaluation on the strength of the given password. User can validate how strong their existing passwords are. HaveIBeenPwned API was used for strength checking.

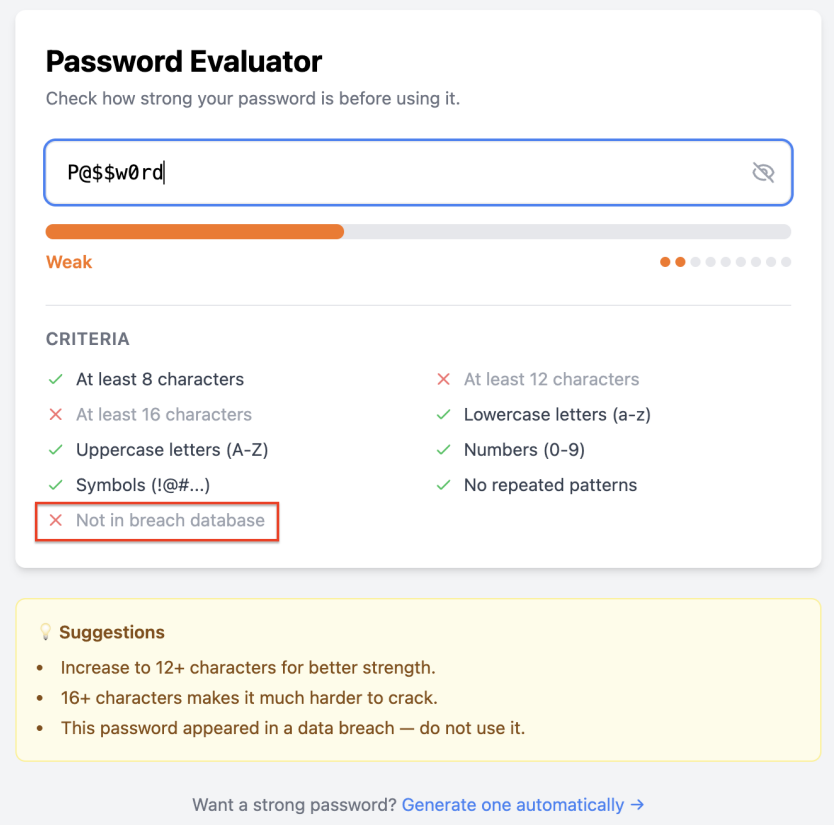
2. Password Generator

Customizable strong password generation with length, character sets, and pattern options. Cryptographically secure random generation.

3. Password History

History of generated passwords with a search functionality. User can also view the full password after the account password is verified again.

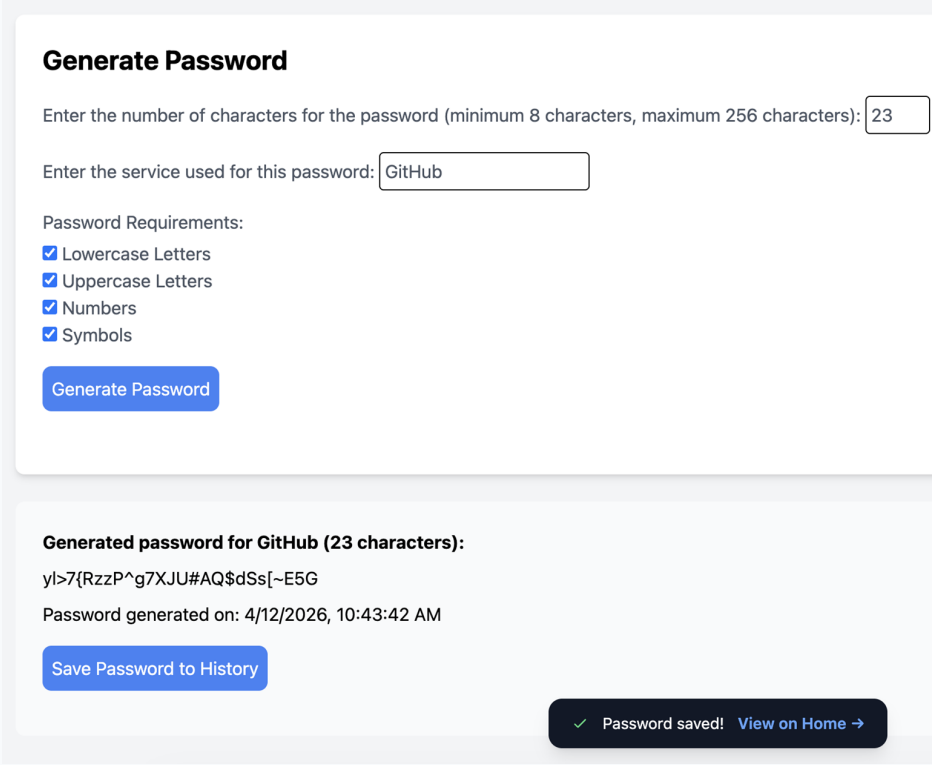
Password Evaluator



- Check how strong your password is
- Strength: Very Weak/Weak/Fair/Strong/Very Strong (based on how many criteria points are met)
- **Have I Been Pwned API** (Third-party API we used):
  - Check password breaches
  - Flags commonly used passwords known to hackers
  - Overrides to ‘Weak’ even if all strength criteria are met

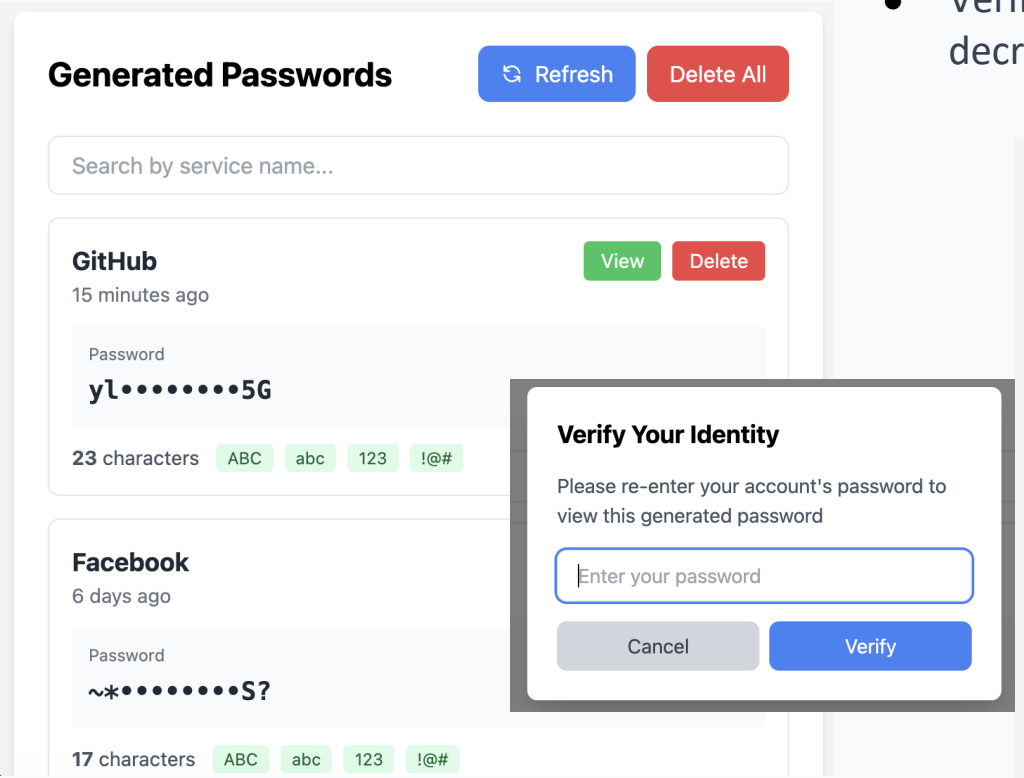
Password Generator

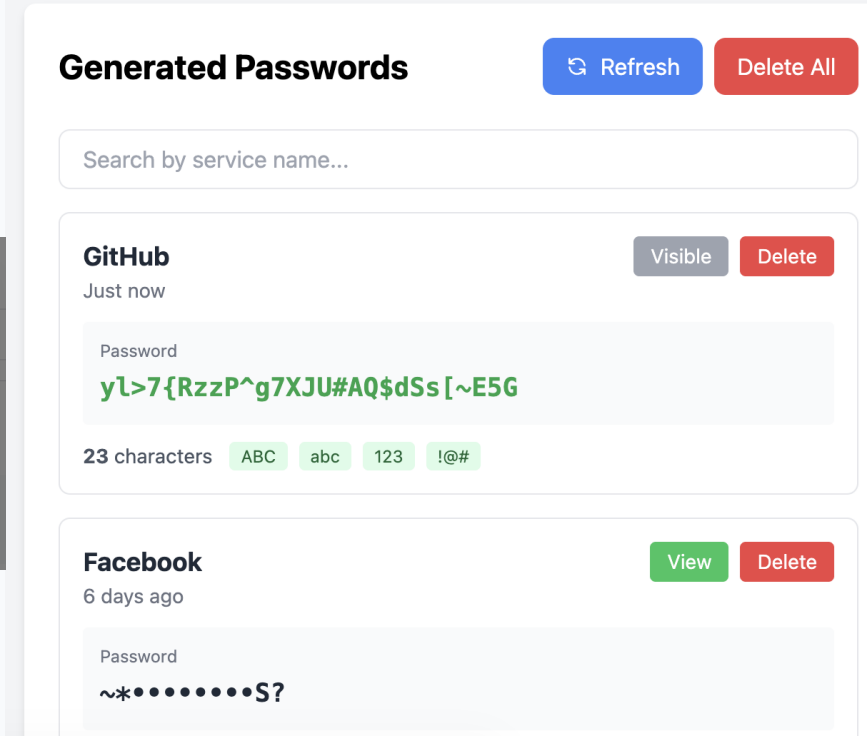
- Generate a new secure password
- Customize password requirements that are required in different services (e.g. GitHub, Amazon)
- Save the newly generated password history if you like it
    - We use **encryption** to avoid storing passwords directly (in AWS DynamoDB)



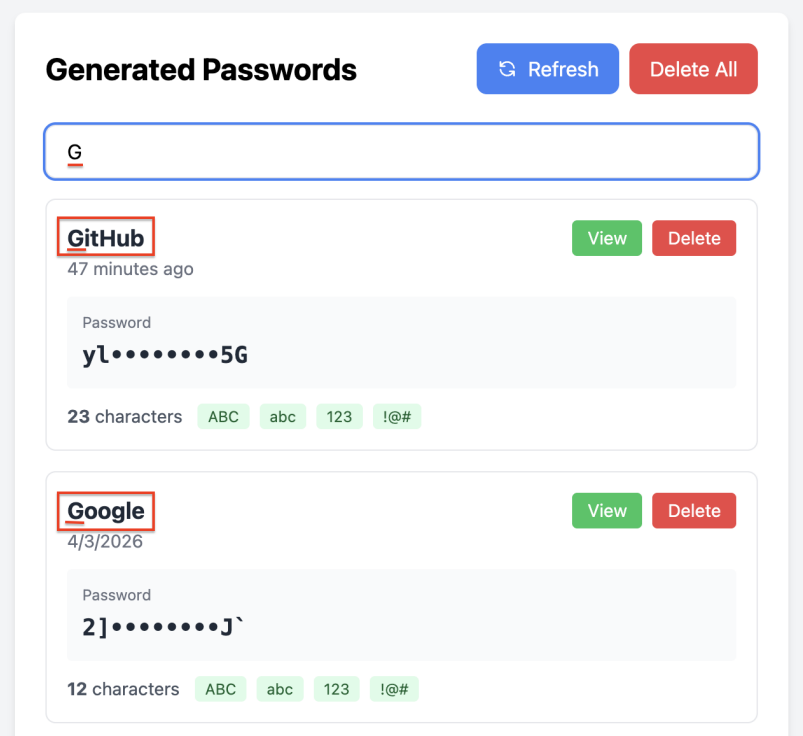
Password History

- Manage passwords that are used in different services
- Verify with your account password to **reveal** the decrypted password that is used in the specific service



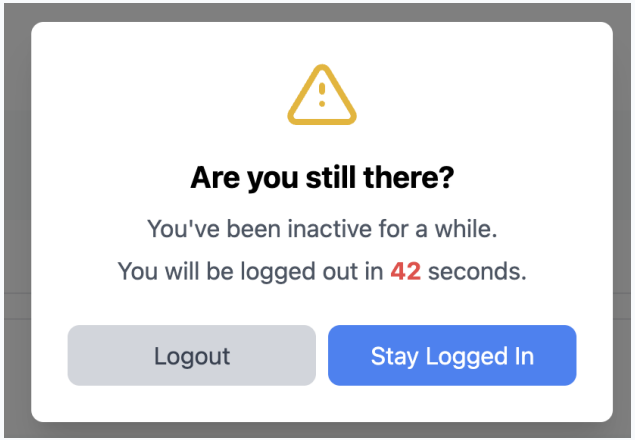


Password History (cont.)



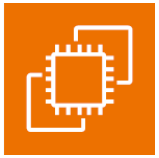
- You can search by service

Idle Session



- Strengthen security

AWS Services



Amazon EC2

- Used for hosting the app



Amazon Cognito

- Used for user authentication



Amazon DynamoDB

- Used for storing users' generated passwords
- Offers fast performance, automatic scaling, and built-in encryption at rest

AWS Services (cont.)



Elastic Load Balancing

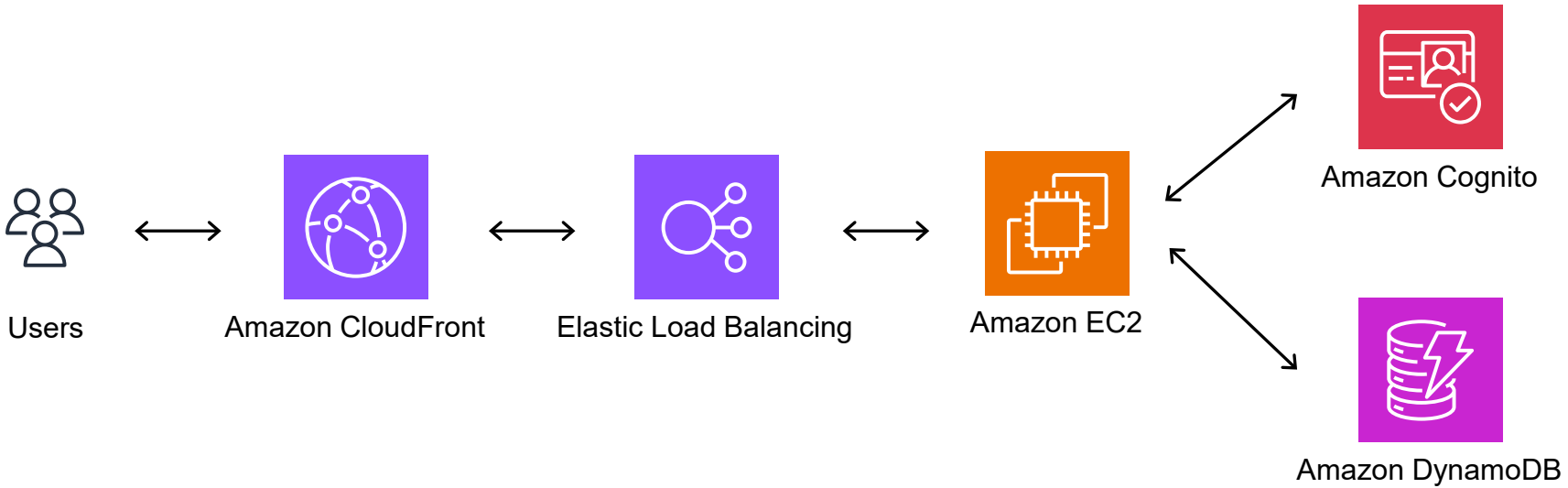
- Enables the communication to the EC2 in a private subnet



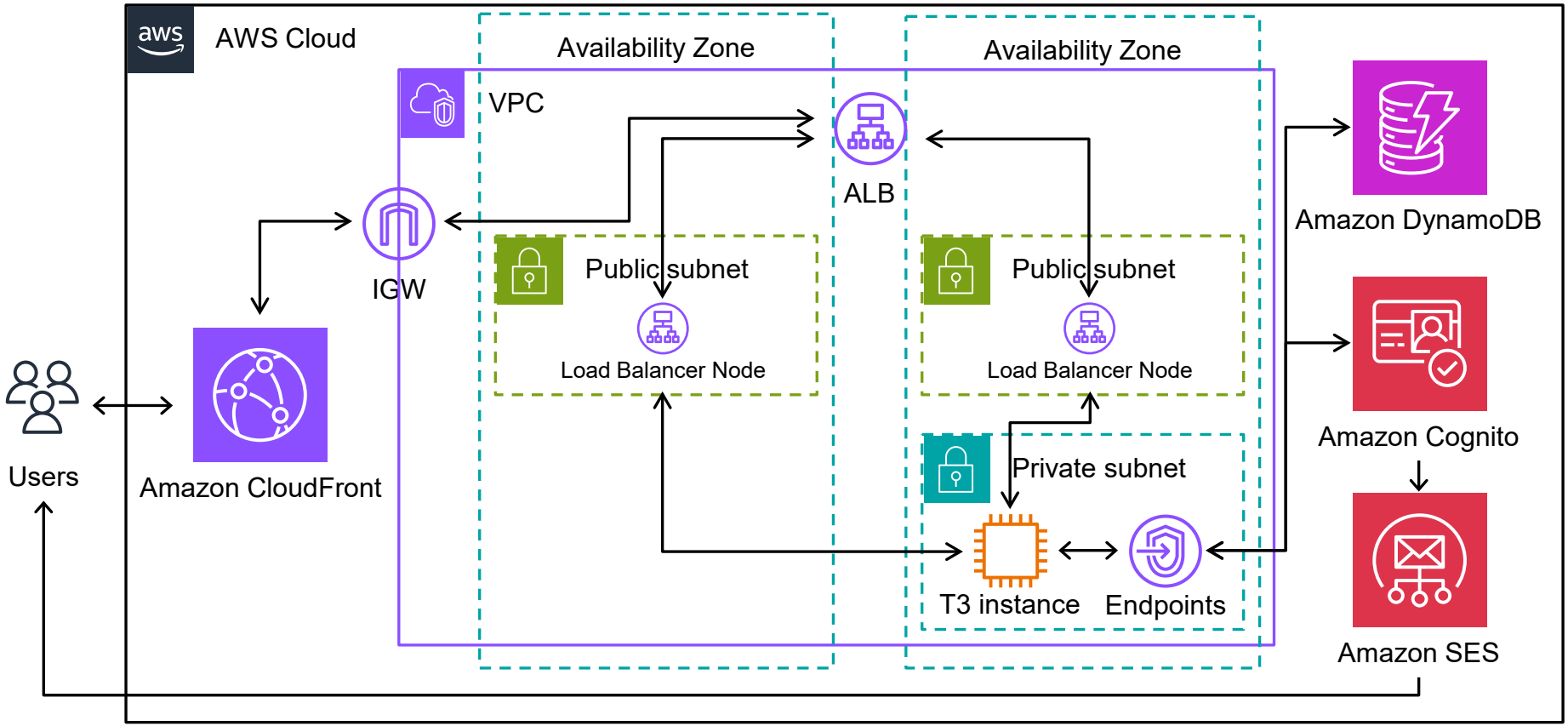
Amazon CloudFront

- Supports HTTPS & DDoS protection

AWS Services Architecture - Overview



AWS Services Architecture - Detail



Challenges

- **Security concern for storing passwords**
  - **Challenge**: For the purpose of our app, we need to store users' passwords. This raises a huge security concern if not handled properly.
  - **Our approach**: We encrypt each password on the client side before sending it to DynamoDB to store. This way, even AWS services don't see the actual passwords. When the user wants to see the full password, we decrypt the password on the client side as well. For the encryption/decryption process, we use CryptoJS library with a secret key that is created using the user's account email address and account password. The secret key is saved in user's session storage, and it's cleared out everytime the user logs out / closes the app as well.

Challenges (cont.)

- **Accessing DynamoDB from EC2**
  - **Challenge**: Our app is hosted on EC2, and we use DynamoDB to store users' passwords. Because EC2 instances don't automatically have permission to talk to DynamoDB, our app is not able to communicate with DynamoDB with the default settings.
  - **Our approach**: We created an IAM Role with appropriate DynamoDB policies and attached it to our EC2 instance. This way, our EC2 now has permission to access the data in DynamoDB.

Challenges (cont.)

- **No Internet access for EC2 in private subnet**
  - **Challenge**: We lauched our EC2 into our private subnet to reduce the chance of getting random hacker attacks by controlling inbound/outbound traffics. However, being in the private subnet means it doesn't have access to the Internet, and users can't access our app.
  - **Our approach**: We used a load balancer as a middle man. The load balancer gets client requests and routes the requests to our EC2. This way users can access our app, without us having to place our EC2 on a public subnet.

Challenges (cont.)

- **Privacy risk for using the 3rd party API**
  - **Challenge**: In Password Evaluator, we need to use and send data to the Have I been Pwned API (the 3rd-party API) to know whether the user input (could be real password in use) has been ever known to hackers. But sending the real password to a 3rd-party API is a privacy risk.
  - **Our approach:** We are able to **mitigate** the privacy risk by using k-anonymity model. Instead of sending the full password, we hash it with SHA-1 and only send the first 5 characters of the hash. The API returns all matching hashes, and we check locally, that way the actual password never leaves the user's device.

Challenges (cont.)

- **HTTPS requirement for crypto.subtle property**
  - **Challenge**: crypto.subtle property is used to encrypt the password user input for evaulation, and it requires HTTPS. For HTTPS, we need to get SSL/TLS certificate, which requires a domain name. For a domain name, we couldn't use Route 53 because registering a domain is not supported for a free tier account.
  - **Our approach**: Instead of registering a domain using a third party service, we used CloudFront, which supports HTTPS by default. With CloudFront, our app gets protected from DDoS as well.

Risks & Issues

- **Hosting the app in a single AZ**
  - **Risk**: If a disaster happens in our AZ, we cannot guarantee availability.
  - **Why we can’t solve**: We don’t expect much traffic for the student-level app. So, we decided to **accept** the risk rather than having more instances in other AZs in order to reduce the operational cost.
- **No easy-to-remember domain name for users**
  - **Issue**: Users can’t easily remember our domain name, which could reduce re-visit rate.
  - **Why we can’t solve**: A human-friendly domain requires registering a domain via Route53, which is not available with our free tier account. Purchasing a domain from a third-party provider could be an option as well, but we decided not to do this, to reduce the operational cost.

Risks & Issues (cont.)

- **AWS SES sandbox mode limitation for email verification**
  - **Issue**: User should be able to freely sign up. But with the current settings, only pre-registered emails in AWS SES can receive verification codes for sign up and login since we are in AWS SES sandbox mode.
  - **Why we can't solve**: To fix it, we need to move to SES production mode and verify email domain (CloudFront is for https web domain, not email domain). But to verify, it requires purchasing a domain (e.g. in Route 53, which is not available in free tier.)

Takeaways

- We were able to create an app that lets users evaluate, generate, and manage passwords securely using multiple AWS services.
- We learned that there are so many other services that we couldn't cover during the lectures, and it's worth exploring different options when building an app using AWS services.
- We learned that dealing with sensitive data like passwords require a lot of attention to security settings. Every decision we make could result in a big security risk if we are not mindful of them.
- We learned that having the EC2 instance in a private subnet (with no internet access) increases the amount of set-up for other parts of the architecrure significantly. For example, we needed to use a VPC endpoint to allow our EC2 to communicate with DynamoDB.
- When we drew the AWS architecture diagram, we learned that DynamoDB and Cognito are managed by AWS, so they exist outside of the VPC we created.

Future plan

- We could improve the storage capacity for generated passwords for cases where the user wants to store hundreds or even thousands of passwords.
- We would like to find a way to get the password requirements from popular services, such as Gmail and Netflix, and apply them to our password generator, so that users don’t have to spend time manually customizing password requirements once they inputted the service name.
- We would like to find a way to get free or cheap email domain to have valid email addresses so that the admins don’t have to manually pre-register users’ email addresses in AWS SES console.
- We would like to learn how to develop a Chrome extension for our app and **automate** the password + service entry process, so that users don’t have to manually visit our app every time. It would automatically detect when a user signs up or logs in to a new service and asks whether they want to store the password in our app.

Demo Time!

https://d2ibn2iy9d52e5.cloudfront.net/



Thank You

Password Manager - Final Presentation